EE587 - Individual Project
Milestone 2
Sobel Edge Detection on Basys 3 FPGA

Sithum Sathsara Vitharana
E/21/420

1 Project Objective

The objective of this project, aligning with EE587 Milestone 2, was to implement a hardware-based Sobel edge detection filter on an Artix-7 FPGA (Basys 3 board) and display the processed image via a VGA interface. Building upon the behavioral simulations and reference generation completed using high-level languages in Milestone 1, this phase transitions the design into physical hardware. The system was designed to act as a standalone hardware accelerator, processing pre-loaded image data entirely within the FPGA silicon and displaying a side-by-side comparison of the original and edge-detected images without distortion.

2 System Architecture & Data Pipeline

The system operates as a completely self-contained hardware accelerator. The laptop is only used once to format the data; all convolution mathematics are performed on the FPGA silicon.

The data pipeline consists of the following stages:

1. **Data Preparation (Python):** A Python script crops and resizes the source image to a 320×480 resolution and converts it into a raw hexadecimal text file (`.hex`).
2. **Memory Storage (`image_rom.v`):** The hex data is permanently flashed into the FPGA's internal Block RAM (BRAM) during compilation.
3. **Pixel Alignment (`pixel_window.v` & `output_row_buffer.v`):** The hardware reads the pixels and shifts them into synchronous buffers to create the 3×3 sliding window required for edge detection.
4. **Mathematical Processing (`sobel_calc.v`):** The convolution module applies the horizontal and vertical Sobel kernel operators using hardware adders and bit-shifters.
5. **Display Synchronization (`vga_controller.v` & `top_sobel_display.v`):** The master controller generates standard VGA timing signals and splits the 640×480 screen exactly in half, routing the raw image to the left and the Sobel output to the right.

3 Memory Architecture: Challenges & Improvements

The primary challenge during hardware implementation was the Basys 3 board's physical memory limit of 225 KB (Block RAM). A full-screen 640×480 8-bit grayscale image requires over 307 KB, exceeding the hardware's capacity. While a full-screen display is possible by aggressively downgrading the image to a 1-bit monochrome depth, I chose to preserve the higher fidelity of an 8-bit image. This required designing a 320×480 portrait layout, allowing both images to comfortably fit in memory (153.6 KB) for an undistorted side-by-side comparison.

Approach A: Vivado IP and .coe Files
Initially, I used the Vivado Block Memory Generator IP wizard to load a `.coe` file. However, processing large data arrays through the Vivado GUI caused severe synthesis hangs and unresponsiveness.

Approach B: Pure Verilog and $readmemh
To resolve this, I pivoted to a pure Verilog architecture. I wrote a Python script to crop and convert the source image into a raw hex file (`.hex`). I then used the Verilog `$readmemh` function alongside `(* rom_style = "block" *)` synthesis attributes to bypass the IP wizard entirely. This method cleanly inferred native BRAM blocks, eliminated synthesis hangs, and provided complete control over the memory architecture.

4 Results

The hardware was successfully synthesized and programmed onto the Basys 3 board. The VGA output demonstrates a flawless split-screen implementation. The left side displays the original grayscale image, while the right side displays the hardware-calculated Sobel edges. The boundary is perfectly aligned with no pixel duplication (ghosting), and the image fills the entire vertical height of the monitor while maintaining the correct aspect ratio.



Figure 1: Side-by-side VGA output showing the original image and the hardware-calculated Sobel edges.

A Appendix: Source Code

A.1 Python Formatting Script (generate_hex.py)

```python
import cv2
import numpy as np

def create_center_cropped_hex(image_path, output_hex_path):
    # 1. Load the original image in grayscale (0 flag)
    img = cv2.imread(image_path, 0)

    if img is None:
        print(f"Error: Could not open or find the image '{image_path}'")
        return

    orig_h, orig_w = img.shape
    print(f"Original image dimensions: {orig_w}x{orig_h}")

    # 2. Calculate center-crop coordinates for a target 320x480 (3:4 aspect ratio)
    target_aspect = 320.0 / 480.0  # 0.6667
    orig_aspect = float(orig_w) / float(orig_h)

    if orig_aspect > target_aspect:
        # Image is too wide: match height, crop width sides
        new_w = int(orig_h * target_aspect)
        new_h = orig_h
        x_offset = (orig_w - new_w) // 2
        y_offset = 0
    else:
        # Image is too tall: match width, crop top/bottom sides
        new_w = orig_w
        new_h = int(orig_w / target_aspect)
        x_offset = 0
        y_offset = (orig_h - new_h) // 2

    # Crop out the calculated box
    cropped_img = img[y_offset:y_offset+new_h, x_offset:x_offset+new_w]

    # 3. Downsample to the exact target hardware boundaries
    final_img = cv2.resize(cropped_img, (320, 480), interpolation=cv2.INTER_AREA)

    # Flatten the 2D array into a 1D sequence of bytes
    pixels = final_img.flatten()

    # Hardware resource sanity check verification
    print(f"Cropped region dimensions: {new_w}x{new_h}")
    print(f"Resized image dimensions: {final_img.shape[1]}x{final_img.shape[0]}")
    print(f"Total elements to write: {len(pixels)}")

    if len(pixels) != 153600:
        print("CRITICAL ERROR: Pixel count is incorrect. Halting generation.")
        return
```

```
    # 4. Generate the plain text hex file ($readmemh format)
    with open(output_hex_path, "w") as f:
        for p in pixels:
            f.write(f"{p:02x}\n")

    print(f"Success! '{output_hex_path}' generated for deployment.")

# Execute script (Make sure 'input_image.png' or your photo file is in the same folder)
create_center_cropped_hex("input_image.png", "image_data.hex")
```

A.2 Top Level Wrapper (top_sobel_display.v)

```
`timescale 1ns / 1ps
module top_sobel_display (
    input clk_100MHz,
    input reset,
    output h_sync,
    output v_sync,
    output [3:0] vga_red,
    output [3:0] vga_green,
    output [3:0] vga_blue
);

    // ---- 100MHz -> 25MHz pixel clock ----
    reg pixel_clk = 0;
    reg [1:0] clk_div = 0;
    always @(posedge clk_100MHz) begin
        clk_div <= clk_div + 1;
        pixel_clk <= clk_div[1];
    end

    // ---- VGA timing ----
    wire video_on;
    wire [9:0] pixel_x;
    wire [9:0] pixel_y;

    vga_controller vga_inst (
        .clk_25MHz(pixel_clk),
        .reset(reset),
        .h_sync(h_sync),
        .v_sync(v_sync),
        .video_on(video_on),
        .pixel_x(pixel_x),
        .pixel_y(pixel_y)
    );

    // ---- Fetch image data only for the valid 320 x 480 region ----
    wire fetch_enable = (pixel_x < 10'd320) && (pixel_y < 10'd480);
    wire [9:0] fetch_row = (pixel_y < 10'd480) ? pixel_y : 10'd479;
    wire [9:0] fetch_col = (pixel_x < 10'd320) ? pixel_x : 10'd319;
    wire [16:0] rom_addr = fetch_row * 320 + fetch_col;

    wire [7:0] raw_pixel;
    image_rom rom_inst (
        .clka(pixel_clk),
```

```
        .addra(rom_addr),
        .douta(raw_pixel)
    );

    // ---- 3x3 sliding window ----
    wire [7:0] p11, p12, p13, p21, p22, p23, p31, p32, p33;
    pixel_window window_inst (
        .clk(pixel_clk),
        .enable(fetch_enable),
        .reset(reset),
        .pixel_in(raw_pixel),
        .p11(p11), .p12(p12), .p13(p13),
        .p21(p21), .p22(p22), .p23(p23),
        .p31(p31), .p32(p32), .p33(p33)
    );

    // ---- Sobel math ----
    wire [7:0] sobel_pixel;
    sobel_calc sobel_inst (
        .clk(pixel_clk),
        .p11(p11), .p12(p12), .p13(p13),
        .p21(p21), .p22(p22), .p23(p23),
        .p31(p31), .p32(p32), .p33(p33),
        .sobel_out(sobel_pixel)
    );

    // ---- Row buffer: write during left half, read during right half ----
    // Pipeline delay = 1 cycle (pixel_window latch) + 4 cycles (sobel_calc) = 5 cycles.
    wire write_en = (pixel_x >= 10'd5) && (pixel_x <= 10'd324) && (pixel_y < 10'd480);
    wire [9:0] write_addr = pixel_x - 10'd5;
    wire [9:0] read_addr  = (pixel_x >= 10'd320 && pixel_x < 10'd640) ? (pixel_x - 10'd320) : 10'd0;
    wire [7:0] edge_pixel;

    output_row_buffer rowbuf_inst (
        .clk(pixel_clk),
        .write_en(write_en),
        .write_addr(write_addr),
        .write_data(sobel_pixel),
        .read_addr(read_addr),
        .read_data(edge_pixel)
    );

    // ---- Final pixel: left = live original, right = edge map of the row just above ----
    wire [7:0] final_pixel = (pixel_x < 10'd320) ? raw_pixel : edge_pixel;
    wire valid_frame = video_on && (pixel_y < 10'd480);

    assign vga_red   = valid_frame ? final_pixel[7:4] : 4'h0;
    assign vga_green = valid_frame ? final_pixel[7:4] : 4'h0;
    assign vga_blue  = valid_frame ? final_pixel[7:4] : 4'h0;

endmodule
```

A.3 VGA Controller (vga_controller.v)

```
`timescale 1ns / 1ps
module vga_controller (
    input clk_25MHz,
    input reset,
    output reg h_sync,
    output reg v_sync,
    output video_on,
    output [9:0] pixel_x,
    output [9:0] pixel_y
);
    parameter HD = 640;
    parameter HF = 16;
    parameter HB = 48;
    parameter HR = 96;
    parameter MAX_H = HD + HF + HB + HR - 1;
    parameter VD = 480;
    parameter VF = 10;
    parameter VB = 33;
    parameter VR = 2;
    parameter MAX_V = VD + VF + VB + VR - 1;

    reg [9:0] h_count = 0;
    reg [9:0] v_count = 0;

    always @(posedge clk_25MHz) begin
        if (reset) begin
            h_count <= 0;
            v_count <= 0;
        end else begin
            if (h_count == MAX_H) begin
                h_count <= 0;
                if (v_count == MAX_V)
                    v_count <= 0;
                else
                    v_count <= v_count + 1;
            end else begin
                h_count <= h_count + 1;
            end
        end
    end

    always @(posedge clk_25MHz) begin
        h_sync <= ~((h_count >= (HD + HF)) && (h_count < (HD + HF + HR)
   ));
        v_sync <= ~((v_count >= (VD + VF)) && (v_count < (VD + VF + VR)
   ));
    end

    assign video_on = (h_count < HD) && (v_count < VD);
    assign pixel_x  = h_count;
    assign pixel_y  = v_count;
endmodule
```

A.4 Image Memory (image_rom.v)

```
`timescale 1ns / 1ps
module image_rom (
    input clka,
    input [16:0] addra,
    output reg [7:0] douta
);
    // 320 x 480 = 153,600 pixels
    (* rom_style = "block" *)
    reg [7:0] rom_memory [0:153599];
    initial begin
        // Use the FULL path to your hex file, not just the filename.
        // Vivado runs synthesis from deep inside a .runs\synth_1 folder,
        // so a plain filename won't be found there.
        $readmemh("C:/Users/Sithum Vitharana/Desktop/Sobel_Filter Milestone_2/Sobel_Filter M2/image_data.hex", rom_memory);
    end
    always @(posedge clka) begin
        douta <= rom_memory[addra];
    end
endmodule
```

A.5 Pixel Window Aligner (pixel_window.v)

```
`timescale 1ns / 1ps

module pixel_window (
    input clk,
    input enable,
    input reset,
    input [7:0] pixel_in,
    output reg [7:0] p11, p12, p13,
    output reg [7:0] p21, p22, p23,
    output reg [7:0] p31, p32, p33
);
    reg [7:0] row1 [0:319];
    reg [7:0] row2 [0:319];
    integer i;

    always @(posedge clk) begin
        if (reset) begin
            p11 <= 0; p12 <= 0; p13 <= 0;
            p21 <= 0; p22 <= 0; p23 <= 0;
            p31 <= 0; p32 <= 0; p33 <= 0;
        end else if (enable) begin
            row1[0] <= pixel_in;
            row2[0] <= row1[319];
            for (i = 1; i < 320; i = i + 1) begin
                row1[i] <= row1[i-1];
                row2[i] <= row2[i-1];
            end
            p11 <= row2[319]; p12 <= row2[318]; p13 <= row2[317];
            p21 <= row1[319]; p22 <= row1[318]; p23 <= row1[317];
            p31 <= pixel_in;  p32 <= row1[0];   p33 <= row1[1];
        end
```

```
    end
endmodule
```

A.6 Output Row Buffer (output_row_buffer.v)

```
`timescale 1ns / 1ps

module output_row_buffer (
    input clk,
    input write_en,
    input [9:0] write_addr,
    input [7:0] write_data,
    input [9:0] read_addr,
    output [7:0] read_data
);
    (* ram_style = "distributed" *)
    reg [7:0] mem [0:319];
    integer k;
    initial begin
        for (k = 0; k < 320; k = k + 1)
            mem[k] = 8'd0;
    end

    always @(posedge clk) begin
        if (write_en)
            mem[write_addr] <= write_data;
    end

    assign read_data = mem[read_addr];
endmodule
```

A.7 Sobel Calculator (sobel_calc.v)

```
`timescale 1ns / 1ps
module sobel_calc (
    input clk,
    input [7:0] p11, p12, p13,
    input [7:0] p21, p22, p23,
    input [7:0] p31, p32, p33,
    output reg [7:0] sobel_out
);
    reg signed [10:0] Gx;
    reg signed [10:0] Gy;
    reg [10:0] abs_Gx;
    reg [10:0] abs_Gy;
    reg [11:0] sum_G;

    always @(posedge clk) begin
        Gx <= ((p13 + (p23 << 1) + p33) - (p11 + (p21 << 1) + p31));
        Gy <= ((p31 + (p32 << 1) + p33) - (p11 + (p12 << 1) + p13));
        abs_Gx <= (Gx[10] == 1'b1) ? -Gx : Gx;
        abs_Gy <= (Gy[10] == 1'b1) ? -Gy : Gy;
        sum_G  <= abs_Gx + abs_Gy;
        if (sum_G > 11'hFF)
            sobel_out <= 8'hFF;
```

```
        else
            sobel_out <= sum_G[7:0];
    end
endmodule
```

A.8 Constraints File (Basys3_Master.xdc)

```
# Clock signal
set_property PACKAGE_PIN W5 [get_ports clk_100MHz]
set_property IOSTANDARD LVCMOS33 [get_ports clk_100MHz]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [
    get_ports clk_100MHz]

# Buttons
# Using the Center Button (btnC) as our hardware system reset
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

# VGA Connector
set_property PACKAGE_PIN G19 [get_ports {vga_red[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_red[0]}]
set_property PACKAGE_PIN H19 [get_ports {vga_red[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_red[1]}]
set_property PACKAGE_PIN J19 [get_ports {vga_red[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_red[2]}]
set_property PACKAGE_PIN N19 [get_ports {vga_red[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_red[3]}]

set_property PACKAGE_PIN J17 [get_ports {vga_green[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_green[0]}]
set_property PACKAGE_PIN H17 [get_ports {vga_green[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_green[1]}]
set_property PACKAGE_PIN G17 [get_ports {vga_green[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_green[2]}]
set_property PACKAGE_PIN D17 [get_ports {vga_green[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_green[3]}]

set_property PACKAGE_PIN N18 [get_ports {vga_blue[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_blue[0]}]
set_property PACKAGE_PIN L18 [get_ports {vga_blue[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_blue[1]}]
set_property PACKAGE_PIN K18 [get_ports {vga_blue[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_blue[2]}]
set_property PACKAGE_PIN J18 [get_ports {vga_blue[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_blue[3]}]

set_property PACKAGE_PIN P19 [get_ports h_sync]
set_property IOSTANDARD LVCMOS33 [get_ports h_sync]
set_property PACKAGE_PIN R19 [get_ports v_sync]
set_property IOSTANDARD LVCMOS33 [get_ports v_sync]
```